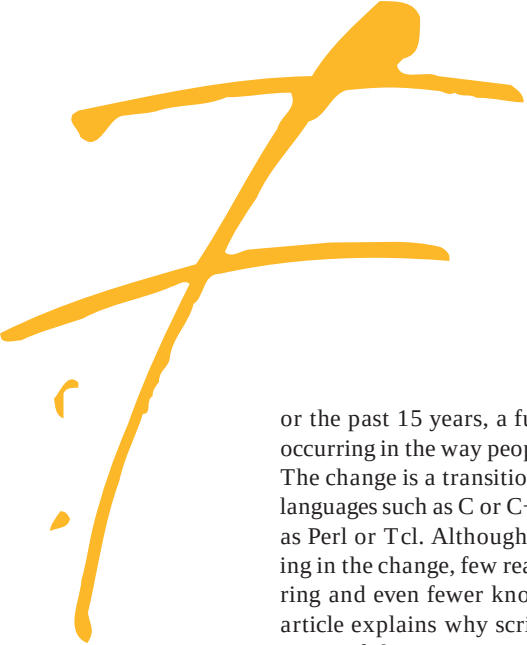


John K. Ousterhout
Sun Microsystems Laboratories

Scripting: Higher-Level Programming for the 21st Century



Increases in computer speed and changes in the application mix are making scripting languages more and more important for the applications of the future. Scripting languages differ from system programming languages in that they are designed for “gluing” applications together. They use typeless approaches to achieve a higher level of programming and more rapid application development than system programming languages.

or the past 15 years, a fundamental change has been occurring in the way people write computer programs. The change is a transition from system programming languages such as C or C++ to scripting languages such as Perl or Tcl. Although many people are participating in the change, few realize that the change is occurring and even fewer know why it is happening. This article explains why scripting languages will handle many of the programming tasks in the next century better than system programming languages.

Scripting languages are designed for different tasks than are system programming languages, and this leads to fundamental differences in the languages. System programming languages were designed for building data structures and algorithms from scratch, starting from the most primitive computer elements such as words of memory. In contrast, scripting languages are designed for gluing: They assume the existence of a set of powerful components and are intended primarily for connecting components. System programming languages are strongly typed to help manage complexity, while scripting languages are typeless to simplify connections among components and provide rapid application development.

Scripting languages and system programming languages are complementary, and most major computing platforms since the 1960s have included both kinds of languages. The languages are typically used together in component frameworks, where components are cre-

ated with system programming languages and glued together with scripting languages. However, several recent trends, such as faster machines, better scripting languages, the increasing importance of graphical user interfaces (GUIs) and component architectures, and the growth of the Internet, have greatly expanded the applicability of scripting languages. These trends will continue over the next decade, with more and more new applications written entirely in scripting languages and system programming languages used primarily for creating components.

SYSTEM PROGRAMMING LANGUAGES

To understand the differences between scripting languages and system programming languages, it is important to understand how system programming languages evolved. System programming languages were introduced as an alternative to assembly languages. In assembly languages, virtually every aspect of the machine is reflected in the program. Each statement represents a single machine instruction and programmers must deal with low-level details such as register allocation and procedure-calling sequences. As a result, it is difficult to write and maintain large programs in assembly languages.

By the late 1950s, higher-level languages such as Lisp, Fortran, and Algol began to appear. In these languages, statements no longer correspond exactly to machine instructions; a compiler translates each state-

Scripting languages assume that a collection of useful components already exist in other languages. They are intended not for writing applications from scratch but rather for combining components.

ment in the source program into a sequence of binary instructions. Over time a series of *system programming languages* evolved from Algol, including PL/1, Pascal, C, C++, and Java. System programming languages are less efficient than assembly languages but they allow applications to be developed much more quickly. As a result, system programming languages have almost completely replaced assembly languages for the development of large applications.

Higher-level languages

System programming languages differ from assembly languages in two ways: they are higher level and they are strongly typed. The term “higher level” means that many details are handled automatically, so programmers can write less code to get the same job done. For example:

- Register allocation is handled by the compiler so that programmers need not write code to move information between registers and memory.
- Procedure calling sequences are generated automatically; programmers need not worry about moving arguments to and from the call stack.
- Programmers can use simple keywords such as `while` and `if` for control structures; the compiler generates all the detailed instructions to implement the control structures.

On average, each line of code in a system programming language translates to about five machine instructions, compared with one instruction per line in an assembly program. (In an informal analysis of eight C files written by five different people, I found that the ratio ranged from three to seven instructions per line;¹ in a study of numerous languages, Capers Jones found that, for a given task, assembly languages require three to six times as many lines of code as system programming languages.²) Programmers can write roughly the same number of lines of code per year regardless of language,³ so system programming languages allow applications to be written much more quickly than assembly languages.

Typing

The second difference between assembly languages and system programming languages is typing. I use the term *typing* to refer to the degree to which the meaning of information is specified in advance of its use. In a strongly typed language, the programmer declares how each piece of information will be used, and the language prevents the information from being used in any other way. In a weakly typed language, there are no a priori restrictions on how information can be used; the meaning of information is determined

solely by the way it is used, not by any initial promises.

Modern computers are fundamentally typeless. Any word in memory can hold any kind of value, such as an integer, a floating-point number, a pointer, or an instruction. The meaning of a value is determined by how it is used. If the program counter points at a word of memory then it is treated as an instruction; if a word is referenced by an integer add instruction, then it is treated as an integer; and so on. The same word can be used in different ways at different times.

In contrast, today’s system programming languages are strongly typed. For example:

- Each variable in a system programming language must be declared with a particular type such as integer or pointer to string, and it must be used in ways that are appropriate for the type.
- Data and code are segregated; it is difficult if not impossible to create new code on the fly.
- Variables can be collected into structures or objects with well-defined substructure and procedures or methods to manipulate them. An object of one type cannot be used where an object of a different type is expected.

Typing has several advantages. First, it makes large programs more manageable by clarifying how things are used and differentiating among things that must be treated differently. Second, compilers use type information to detect certain kinds of errors, such as an attempt to use a floating-point value as a pointer. Third, typing improves performance by allowing compilers to generate specialized code. For example, if a compiler knows that a variable always holds an integer value, then it can generate integer instructions to manipulate the variable; if the compiler does not know the type of a variable, then it must generate additional instructions to check the variable’s type at runtime.

Figure 1 compares a variety of languages on the basis of their level of programming and strength of typing.

SCRIPTING LANGUAGES

Scripting languages such as Perl,⁴ Python,⁵ Rexx,⁶ Tcl,⁷ Visual Basic, and the Unix shells represent a very different style of programming than do system programming languages. Scripting languages assume that a collection of useful components already exist in other languages. They are intended not for writing applications from scratch but rather for combining components. For example, Tcl and Visual Basic can be used to arrange collections of user interface controls on the screen, and Unix shell scripts are used to assemble filter programs into pipelines. Scripting languages are often used to extend the features of components; however, they are rarely used for complex algorithms and

data structures, which are usually provided by the components. Scripting languages are sometimes referred to as *glue languages* or *system integration languages*.

Scripting languages are generally typeless

To simplify the task of connecting components, scripting languages tend to be typeless. All things look and behave the same so that they are interchangeable. For example, in Tcl or Visual Basic a variable can hold a string one moment and an integer the next. Code and data are often interchangeable, so that a program can write another program and then execute it on the fly. Scripting languages are often string-oriented, as this provides a uniform representation for many different things.

A typeless language makes it much easier to hook together components. There are no a priori restrictions on how things can be used, and all components and values are represented in a uniform fashion. Thus any component or value can be used in any situation; components designed for one purpose can be used for totally different purposes never foreseen by the designer. For example, in the Unix shells all filter programs read a stream of bytes from an input and write a stream of bytes to an output. Any two programs can be connected by attaching the output of one program to the input of the other. The following shell command stacks three filters together to count the number of lines in the selection that contain the word “scripting”:

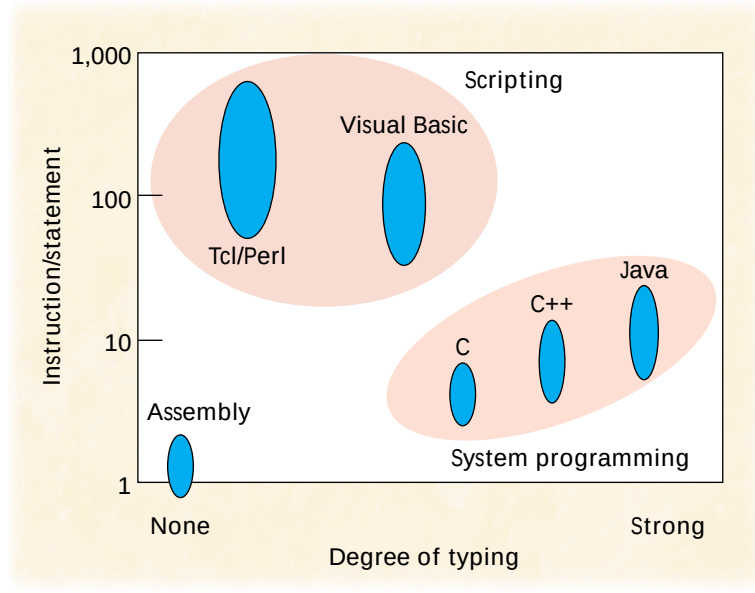
```
select | grep scripting | wc
```

The `select` program reads the text that is currently selected on the display and prints the text on its output; the `grep` program reads its input and prints on its output the lines containing “scripting”; the `wc` program counts the number of lines on its input. Each of these programs can be used in numerous other situations to perform different tasks.

The strongly typed nature of system programming languages discourages reuse. It encourages programmers to create a variety of incompatible interfaces, each of which requires objects of specific types. The compiler prevents any other types of objects from being used with the interface, even if that would be useful. So to use a new object with an existing interface, the programmer must write conversion code to translate between the type of the object and the type expected by the interface. This in turn requires recompiling part or all of the application; many applications today are distributed in binary form so this is not possible.

To appreciate the advantages of a typeless language, consider the following Tcl command:

```
button .b -text Hello! -font {Times
    16} -command {puts hello}
```



This command creates a new button control that displays a text string in a 16-point Times font and prints a short message when the user clicks on the control. The command mixes six different types of things in a single statement: a command name (`button`), a button control (`.b`), property names (`-text`, `-font`, and `-command`), simple strings (`Hello!` and `hello`), a font name (`Times 16`) that includes a typeface name (`Times`) and a size in points (`16`), and a Tcl script (`puts hello`). Tcl represents all of these things uniformly with strings. In this example, the properties can be specified in any order and unspecified properties are given default values; more than 20 properties were left unspecified in the example.

The same example requires seven lines of code in two methods when implemented in Java. With C++ and Microsoft Foundation Classes (MFC), it requires about 25 lines of code in three procedures.¹ Just setting the font requires several lines of code in MFC:

```
CFont *fontPtr = new CFont();
fontPtr->CreateFont(16, 0, 0, 0, 700,
    0, 0, 0, ANSI_CHARSET,
    OUT_DEFAULT_PRECIS,
    CLIP_DEFAULT_PRECIS,
    DEFAULT_QUALITY,
    DEFAULT_PITCH|FF_DONTCARE,
    "Times New Roman");
buttonPtr->SetFont(fontPtr);
```

Much of this code is a consequence of the strong typing. To set the font of a button, its `SetFont` method must be invoked, but this method must be passed a pointer to a `CFont` object. This in turn requires a new object to be declared and initialized. To initialize the `CFont` object its `CreateFont` method must be

Figure 1. A comparison of various programming languages based on their level (higher-level languages execute more machine instructions for each language statement) and their degree of typing. System programming languages such as C tend to be strongly typed and medium level (five to 10 instructions per statement). Scripting languages such as Tcl tend to be weakly typed and very high level (100 to 1,000 instructions per statement).

It might seem that the typeless nature of scripting languages could allow errors to go undetected, but in practice scripting languages are just as safe as system programming languages.

invoked, but `CreateFont` has a rigid interface that requires 14 different arguments to be specified. In Tcl, the essential characteristics of the font (typeface Times, size 16 points) can be used immediately with no declarations or conversions. Furthermore, Tcl allows the button's behavior to be included directly in the command that creates the button, while C++ and Java require it to be placed in a separately declared method.

(In practice, a trivial example like this would probably be handled with a graphical development environment that hides the complexity of the underlying language. The user enters property values in a form and the development environment outputs the code. However, in more complex situations, such as conditional assignment of property values or interfaces generated programmatically, the developer must write code in the underlying language.)

It might seem that the typeless nature of scripting languages could allow errors to go undetected, but in practice scripting languages are just as safe as system programming languages. For example, an error will occur if the font size specified for the button example above is a noninteger string such as `xyz`. Scripting languages do their error checking at the last possible moment, when a value is used. Strong typing allows errors to be detected at compile time, so the cost of runtime checks is avoided. However, the price to be paid for efficiency is restrictions on how information can be used; this results in more code and less flexible programs.

Scripting languages are interpreted

Another key difference between scripting languages and system programming languages is that scripting languages are usually interpreted, whereas system programming languages are usually compiled. Interpreted languages provide rapid turnaround during development by eliminating compile times. Interpreters also make applications more flexible by allowing users to program the applications at runtime. For example, many synthesis and analysis tools for integrated circuits include a Tcl interpreter; users of the programs write Tcl scripts to specify their designs and control the operation of the tools. Interpreters also allow powerful effects to be achieved by generating code on the fly. For example, a Tcl-based Web browser can parse a Web page by translating the HTML for the page into a Tcl script using a few regular expression substitutions. It then executes the Tcl script to render the page on the screen.

Scripting languages are less efficient than system programming languages, in part because they use interpreters instead of compilers but also because their basic components are chosen for power and ease of use rather than an efficient mapping onto the underlying hardware. For example, scripting languages often use variable-

length strings in situations where a system programming language would use a binary value that fits in a single machine word, and they use hash tables where system programming languages use indexed arrays.

Fortunately, the performance of a scripting language is not usually a major issue. Applications for scripting languages are generally smaller than applications for system programming languages, and the performance of a scripting application tends to be dominated by the performance of the components, which are typically implemented in a system programming language.

Scripting languages are higher level than system programming languages in the sense that a single statement does more work on average. A typical statement in a scripting language executes hundreds or thousands of machine instructions, whereas a typical statement in a system programming language executes about five machine instructions (as Figure 1 illustrates). Part of this difference is because scripting languages use interpreters, but much of the difference is because the primitive operations in scripting languages have greater functionality. For example, in Perl it is about as easy to invoke a regular expression substitution as it is to invoke an integer addition. In Tcl, a variable can have traces associated with it so that setting the variable causes side effects; for example, a trace might be used to keep the variable's value updated continuously on the screen.

Scripting languages allow rapid development of gluing-oriented applications. Table 1 provides anecdotal support for this claim. It describes several applications that were implemented in a system programming language and then reimplemented in a scripting language or vice versa. In every case, the scripting version required less code and development time than the system programming version; the difference varied from a factor of two to a factor of 60. Scripting languages provided less benefit when they were used for the first implementation; this suggests that any reimplementation benefits substantially from the experiences of the first implementation and that the true difference between scripting and system programming is more like a factor of five to 10, rather than the extreme points of the table. The benefits of scripting also depend on the application. In the last example in Table 1, the GUI part of the application is gluing-oriented but the simulator part is not; this might explain why the application benefited less from scripting than other applications.

The information in the table was provided by various Tcl developers in response to an article posted on the `comp.lang.tcl` newsgroup.¹

DIFFERENT TOOLS FOR DIFFERENT TASKS

A scripting language is not a replacement for a system programming language or vice versa. Each is

Table 1. Comparing applications implemented in both system programming languages and scripting languages.

Application (contributor)	Comparison	Code ratio*	Effort ratio**	Comments
Database application (Ken Corey)	C++ version: 2 months Tcl version: 1 day		60	C++ version implemented first; Tcl version had more functionality
Computer system test and installation (Andy Belsey)	C test application: 272,000 lines, 120 months C FIS application: 90,000 lines, 60 months Tcl/Perl version: 7,700 lines, 8 months	47	22	C version implemented first; Tcl/Perl version replaced both C applications
Database library (Ken Corey)	C++ version: 2-3 months Tcl version: 1 week		8-12	C++ version implemented first
Security scanner (Jim Graham)	C version: 3,000 lines Tcl version: 300 lines	10		C version implemented first; Tcl version had more functionality
Display oil well production curves (Dan Schenck)	C version: 3 months Tcl version: 2 weeks		6	Tcl version implemented first
Query dispatcher (Paul Healy)	C version: 1,200 lines, 4-8 weeks Tcl version: 500 lines, 1 week	2.5	4-8	C version implemented first, uncommented; Tcl version had comments, more functionality
Spreadsheet tool	C version: 1,460 lines Tcl version: 380 lines	4		Tcl version implemented first
Simulator and GUI (Randy Wang)	Java version: 3,400 lines, 3-4 weeks Tcl version: 1,600 lines, <1 week	2	3-4	Tcl version had 10 to 20 percent more functionality and was implemented first

* Code ratio is the ratio of lines of code for the two implementations (<1 means the system programming language required more lines).

** Effort ratio is the ratio of development times. In most cases the two versions were implemented by different people.

suiting to a different set of tasks. For gluing and system integration, applications can be developed five to 10 times faster with a scripting language; system programming languages require large amounts of boilerplate and conversion code to connect the pieces, whereas this can be done directly with a scripting language. For complex algorithms and data structures, the strong typing of a system programming language makes programs easier to manage. Where execution speed is key, a system programming language can often run 10 to 20 times faster than a scripting language because it makes fewer runtime checks.

In deciding whether to use a scripting language or a system programming language for a particular task, consider the following questions:

- Is the application's main task to connect preexisting components?
- Will the application manipulate a variety of different things?
- Does the application include a GUI?
- Does the application do a lot of string manipulation?
- Will the application's functions evolve rapidly over time?
- Does the application need to be extensible?

Affirmative answers to these questions suggest that a scripting language will work well for the applica-

tion. On the other hand, affirmative answers to the following questions suggest that an application is better suited to a system programming language:

- Does the application implement complex algorithms or data structures?
- Does the application manipulate large datasets, for example, all the pixels in an image, such that execution speed is critical?
- Are the application's functions well defined and slow to change?

Most of the major computing platforms over the past 30 years have provided both system programming and scripting languages. For example, one of the first scripting languages, albeit a crude one, was Job Control Language (JCL), which was used to sequence job steps in OS/360. The individual job steps were written in PL/1, Fortran, or assembler languages, which were the system programming languages of the day. In the Unix machines of the 1980s, C was used for system programming and shell programs such as `sh` and `csh` for scripting. In the PC world of the 1990s, C and C++ are used for system programming and Visual Basic for scripting. In the Internet world that is taking shape now, Java is used for system programming and languages such as JavaScript, Perl, and Tcl are used for scripting.

Scripting and system programming are symbiotic. Used together, they produce programming environments of exceptional power.

Scripting and system programming are symbiotic. Used together, they produce programming environments of exceptional power. System programming languages are used to create exciting components that can then be assembled using scripting languages. For example, much of the attraction of Visual Basic is that system programmers can write ActiveX components in C and less sophisticated programmers can then use the components in Visual Basic applications. In Unix it is easy to write shell scripts that invoke applications written in C. One of the reasons for the popularity of Tcl

is the ability to extend the language by writing C code that implements new commands.

SCRIPTING IS ON THE RISE

Scripting languages have existed for a long time, but in recent years several factors have combined to increase their importance. The most important factor is a shift in the application mix toward gluing applications. Three examples of this shift are GUIs, the Internet, and component frameworks.

Graphical user interfaces

GUIs first appeared in the early 1980s and became widespread by the end of the decade; GUIs now account for half or more of the total effort in many programming projects. GUIs are fundamentally gluing applications. The goal is not to create new functionality but to make connections between a collection of graphical controls and the internal functions of the application.

I am not aware of any rapid-development environments for GUIs based on a system programming language. Whether the environment is Windows, Macintosh Toolbox, or Unix Motif, GUI toolkits based on languages such as C or C++ have proven hard to learn, clumsy to use, and inflexible in the results they produce. Some of these systems have nice graphical tools for designing screen layouts that hide the underlying language, but things become difficult as soon as the designer has to write code, for example, to provide the behaviors for the interface elements. All of the best rapid-development GUI environments are based on scripting languages: Visual Basic, HyperCard, and Tcl/Tk.

Internet

The growth of the Internet has also popularized scripting languages. The Internet is nothing more than a gluing tool. It does not create any new computations or data; it simply makes a huge number of existing things easily accessible. The ideal language for most Internet programming tasks is one that makes it possible for all the connected components to work together; that is, a scripting language. For example,

Perl has become popular for writing CGI scripts and JavaScript is popular for scripting in Web pages.

Component frameworks

The third example of scripting-oriented applications is component frameworks such as ActiveX and JavaBeans. Although system programming languages work well for creating components, the task of assembling components into applications is better suited to scripting. Without a good scripting language to manipulate the components, much of the power of a component framework is lost. This might explain in part why component frameworks have been more successful on PCs, where Visual Basic provides a convenient scripting tool, than on other platforms such as Unix/CORBA, where scripting is not included in the component framework.

Better scripting technology

Another reason for the increasing popularity of scripting languages is improvements in scripting technology. Modern scripting languages such as Tcl and Perl are a far cry from early scripting languages such as JCL. For example, JCL did not even provide basic iteration and early Unix shells did not support procedures. Scripting technology is still relatively immature even today. For example, Visual Basic is not really a scripting language; it was originally implemented as a simple system programming language, then modified to make it more suitable for scripting. Future scripting languages will be even better than those available today.

Scripting technology has also benefited from the everincreasing speed of computer hardware. It used to be that the only way to get acceptable performance in an application of any complexity was to use a system programming language. In some cases, even system programming languages were not efficient enough, so the applications had to be written in assembler. However, machines today are 100 to 500 times faster than the machines of 1980, and performance doubles every 18 months. Today, many applications can be implemented in an interpreted language and still have excellent performance. For example, a Tcl script can manipulate collections with several thousand objects and still provide good interactive response. As computers get faster, scripting will become attractive for larger and larger applications.

Casual programmers

One final reason for the increasing use of scripting languages is a change in the programmer community. Twenty years ago, most programmers were sophisticated programmers working on large projects. Programmers of that era expected to spend several months to master a language and its programming environment, and system programming languages

were designed for such programmers. However, since the arrival of the PC more and more casual programmers have joined the programmer community. For these people, programming is not their main job function; it is a tool they use occasionally to help with their main job. Examples of casual programming are simple database queries or macros for a spreadsheet.

Casual programmers are unwilling to spend months learning a system programming language, but they can often learn enough about a scripting language in a few hours to write useful programs. Scripting languages are easier to learn because they have simpler syntax than system programming languages and they omit complex features like objects and threads. For example, compare Visual Basic with Visual C++; few casual programmers would attempt to use Visual C++, but many have built useful applications with Visual Basic.

Even today, the number of applications written in scripting languages is much greater than the number of applications written in system programming languages. On Unix systems, there are many more shell scripts than C programs, and under Windows there are many more Visual Basic programs and applications than C or C++. Of course, most of the largest and most widely used applications are written in system programming languages, so a comparison on the basis of total lines of code or number of installed copies might still favor system programming languages. Nonetheless, scripting languages are already a major force in application development and their market share will increase in the future.

THE ROLE OF OBJECTS

Scripting languages have been mostly overlooked by experts in programming languages and software engineering. Instead, the experts have focused their attention on object-oriented system programming languages such as C++ and Java. Object-oriented (OO) programming is widely believed to represent the next major step in the evolution of programming languages. OO features such as strong typing and inheritance are often claimed to reduce development time, increase software reuse, and solve many other problems including those addressed by scripting languages.

How much benefit has OO programming actually provided? Unfortunately I have not seen enough quantitative data to answer this question definitively. In my opinion, objects provide only a modest benefit: perhaps a 20 to 30 percent improvement in productivity but certainly not a factor of two, let alone a factor of 10. C++ now seems to be reviled as much as it is loved, and some language experts are beginning to speak out against OO programming.⁸ Objects do not improve productivity in the dramatic way that scripting does, and the benefits of OO programming can be achieved in scripting languages.

OO programming does not provide a large improvement in productivity because it neither raises the level of programming nor encourages reuse. In an OO language such as C++, programmers still work with small basic units that must be described and manipulated in great detail. In principle, powerful library packages could be developed, and if these libraries were used extensively they could raise the level of programming. However, few such libraries exist. The strong typing of most OO languages encourages narrowly defined packages that are hard to reuse. Each package requires objects of a specific type; if two packages are to work together, conversion code must be written to translate between the types required by the packages.

Another problem with OO languages is their emphasis on inheritance. Implementation inheritance, in which one class borrows code that was written for another class, is a bad idea that makes software harder to manage and reuse. It binds the implementations of classes together so that neither class can be understood without the other. A subclass cannot be understood without knowing how the inherited methods are implemented in its superclass, and a superclass cannot be understood without knowing how its methods are inherited in subclasses. In a complex class hierarchy, no individual class can be understood without understanding all the other classes in the hierarchy. Even worse, a class cannot be separated from its hierarchy for reuse. Multiple inheritance makes these problems even worse. Implementation inheritance causes the same intertwining and brittleness that have been observed when `goto` statements are overused. As a result, OO systems often suffer from complexity and lack of reuse.

Scripting languages, on the other hand, have actually generated significant software reuse. The model they follow is one in which interesting components are built in a system programming language and then glued together into applications with a scripting language. This division of labor provides a natural framework for reusability. Components are designed to be reusable, and there are well-defined interfaces between components and scripts that make it easy to use components. For example, in Tcl the components are custom commands implemented in C; they look just like the built-in commands so they are easy to invoke in Tcl scripts. In Visual Basic, the components are ActiveX extensions, which can be used by dragging them from a palette onto a form.

Nonetheless, OO programming does provide at least two useful features. The first is encapsulation: objects combine data and code in a way that hides implementation details. This makes it easier to manage large systems. The second useful feature is interface inheritance, where classes provide the same methods and applica-

Implementation inheritance causes the same intertwining and brittleness that have been observed when `goto` statements are overused. As a result, OO systems often suffer from complexity and lack of reuse.

Raising the level of programming should be the single most important goal for language designers, as it has the greatest effect on programmer productivity. It is not clear that strong typing contributes to this goal.

tion programming interfaces (APIs) even though they have different implementations. This makes the classes interchangeable and encourages reuse.

Fortunately, the benefits of objects can be achieved in scripting languages as well as system programming languages, and virtually all scripting languages have some support for OO programming. For example, Python is an OO scripting language; Perl v5 includes support for objects; Object Rexx is an OO version of Rexx; and Incr Tcl is an OO extension to Tcl. One difference is that objects in scripting languages tend to be typeless, while objects in system programming languages tend to be strongly typed.

OTHER LANGUAGES

This article is not intended as a complete characterization of all programming languages. There are many other attributes of programming languages besides strength of typing and the level of programming, and there are many interesting languages that cannot be characterized cleanly as a system programming language or a scripting language. For example, the Lisp family of languages lies somewhere between scripting and system programming, with some of the attributes of each. Lisp pioneered concepts such as interpretation and dynamic typing that are now common in scripting languages, as well as automatic storage management and integrated development environments, which are now used in both scripting and system programming languages.

Scripting languages represent a different set of trade-offs than system programming languages. They give up execution speed and strength of typing relative to system programming languages but provide significantly higher programmer productivity and software reuse. This trade-off makes more and more sense as computers become faster and cheaper in comparison to programmers. System programming languages are well suited to building components where the complexity is in the data structures and algorithms, while scripting languages are well suited for gluing applications where the complexity is in the connections. Gluing tasks are becoming more and more prevalent, so scripting will become an increasingly important programming paradigm in the next century.

I hope this article will impact the computing community in three ways. First, I hope programmers will consider the differences between scripting and system programming when starting new projects and choose the most powerful tool for each task. Second, I hope designers of component frameworks will recognize the importance of scripting and ensure that their frameworks include not only facilities for creating components but also facilities for gluing them together. Finally,

I hope the programming language research community will shift some of its attention to scripting languages and help develop even more powerful scripting languages. Raising the level of programming should be the single most important goal for language designers, as it has the greatest effect on programmer productivity. It is not clear that strong typing contributes to this goal. ♦

Acknowledgments

This article has benefited from many people's comments, including Joel Bartlett, Bill Eldridge, Jeffrey Haemer, Mark Harrison, Paul McJones, David Patterson, Stephen Uhler, Hank Walker, Chris Wright, the *Computer* referees, and dozens of others who participated in a heated net-news discussion of an early draft. Colin Stevens wrote the MFC version of the button example and Stephen Uhler wrote the Java version.

References

1. J. Ousterhout, "Additional Information for Scripting White Paper," <http://www.scriptics.com/people/john.ousterhout/scriptextra.html>.
2. C. Jones, "Programming Languages Table, Release 8.2," Mar. 1996, <http://www.spr.com/library/0langtbl.htm>.
3. B. Boehm, *Software Engineering Economics*, Prentice Hall, Englewood Cliffs, N.J., 1981.
4. L. Wall, T. Christiansen, and R. Schwartz, *Programming Perl*, 2nd ed., O'Reilly & Associates, Sebastopol, Calif., 1996.
5. M. Lutz, *Programming Python*, O'Reilly & Associates, Sebastopol, Calif., 1996.
6. R. O'Hara and D. Gomberg, *Modern Programming Using REXX*, Prentice Hall, Englewood Cliffs, N.J., 1988.
7. J. Ousterhout, *Tcl and the Tk Toolkit*, Addison-Wesley, Reading, Mass., 1994.
8. S. Johnson, "Objecting To Objects," Invited Talk, Usenix Technical Conf., San Francisco, Jan. 1994.

John K. Ousterhout is the CEO of Scriptics Corp. He was previously a Distinguished Engineer at Sun Microsystems Inc., where the work for this article was carried out. His interests include scripting languages, Internet programming, user interfaces, and operating systems. He created the Tcl scripting language and the Tk toolkit. He received a BS in physics from Yale University and a PhD in computer science from Carnegie Mellon University. Ousterhout is a fellow of the Association for Computing Machinery and has received many awards, including the ACM Grace Murray Hopper Award, the US National Science Foundation Presidential Young Investigator Award, and the University of California at Berkeley Distinguished Teaching Award.

Contact Ousterhout at ouster@scriptics.com.